

ANÁLISIS DEL RETO

Juan Sebastián Rojas Marín, 202610140, js.rojasm123@uniandes.edu.co

Katy Teran Mercado, 202614247, k.teranm@uniandes.edu.co

Juan José Ricon Santos, 202615789, jj.rincons1@uniandes.edu.co

Requerimiento <<5>>

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

```
def req_5(catalog, filtro, product, start_date, end_date):  
    """  
    Retorna el resultado del requerimiento 5  
    """  
    # TODO: Modificar el requerimiento 5  
    start_time = get_time()  
  
    orders = catalog["orders_linked"]  
  
    count = 0  
    sum_price = 0  
    sum_boxes = 0  
    sum_marketing = 0  
  
    selected_order = None  
  
    node = orders["first"]  
  
    while node is not None:  
        order = node["info"]  
  
        if order["Product"] == product:  
            date = order["Order_Date"]  
  
            if start_date <= date <= end_date:  
                count += 1  
  
                price = float(order["Price_per_Box"])  
                boxes = int(order["Boxes_Shipped"])  
                marketing = float(order["Marketing_Spend"])  
                amount = float(order["Amount"])  
  
                sum_price += price  
                sum_boxes += boxes  
                sum_marketing += marketing  
  
                if selected_order is None:  
                    selected_order = order  
  
            else:  
                selected_amount = float(selected_order["Amount"])  
                selected_price = float(selected_order["Price_per_Box"])  
                selected_marketing = float(selected_order["Marketing_Spend"])  
  
                if filtro == "MENOR":  
                    if amount < selected_amount:  
                        selected_order = order
```

```

else:
    selected_amount = float(selected_order["Amount"])
    selected_price = float(selected_order["Price_per_Box"])
    selected_marketing = float(selected_order["Marketing_Spend"])

    if filtro == "MENOR":
        if amount < selected_amount:
            selected_order = order

        elif amount == selected_amount:
            if price < selected_price:
                selected_order = order

            elif price == selected_price:
                if marketing < selected_marketing:
                    selected_order = order

    elif filtro == "MAYOR":
        if amount > selected_amount:
            selected_order = order

        elif amount == selected_amount:
            if price < selected_price:
                selected_order = order

            elif price == selected_price:
                if marketing < selected_marketing:
                    selected_order = order

node = node["next"]

```

```

node = node["next"]

if count > 0:
    avg_price = sum_price / count
    avg_boxes = sum_boxes / count
    avg_marketing = sum_marketing / count
else:
    avg_price = 0
    avg_boxes = 0
    avg_marketing = 0

end_time = get_time()

result = {
    "filter": filtro,
    "count": count,
    "selected_order": selected_order,
    "avg_price": avg_price,
    "avg_boxes": avg_boxes,
    "avg_marketing": avg_marketing
}

return result, delta_time(start_time, end_time)

```

Este requerimiento permite identificar el pedido de menor o mayor Amount de un producto dentro de un rango de fechas. El algoritmo recorre los pedidos almacenados en una lista sencillamente encadenada. Para cada pedido verifica si el producto corresponde al ingresado y si su fecha se encuentra entre la fecha inicial y la fecha final. Cuando cumple ambas condiciones, el pedido se tiene en cuenta para calcular los promedios y para comparar su Amount con el pedido seleccionado hasta ese momento. Si el filtro es "MENOR", se conserva el pedido con menor Amount; si es "MAYOR", se conserva el de mayor Amount. En caso de empate en Amount, se selecciona el pedido con menor Price_per_Box y, si el empate continúa, el de menor Marketing_Spend.

Entrada	Estructuras de datos del modelo, filtro ("MENOR" o "MAYOR"), Product, fecha inicial(start_date) y fecha fina(end_date)
Salidas	Tiempo de ejecución, filtro utilizado, cantidad de pedidos encontrados, pedido seleccionado, promedio de Price_per_Box, promedio de Boxes_Shipped y promedio de Marketing_Spend.
Implementado (Sí/No)	Sí, implementado por todo el grupo

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Obtener el primer nodo de la lista enlazada	$O(1)$
Recorrer los n pedidos de la lista enlazada	$O(n)$
Verificar producto y rango de fechas por pedido	$O(1)$
Actualizar sumas para los promedios por pedido válido	$O(1)$
Comparar Amount y aplicar criterios de desempate	$O(1)$
Calcular los promedios finales	$O(1)$
Construir resultado	$O(1)$
TOTAL	$O(n)$

Análisis

En este requerimiento se recorren los pedidos de la lista enlazada para revisar cuáles corresponden al producto ingresado y están dentro del rango de fechas. Cada pedido se revisa una sola vez y, si cumple las condiciones, se hacen las sumas necesarias para los promedios y las comparaciones para encontrar el pedido de menor o mayor Amount. Como esas operaciones son directas, tienen complejidad $O(1)$ para cada pedido.

Por eso, en el peor de los casos se deben revisar los n pedidos de la lista, haciendo que la complejidad total del requerimiento sea $O(n)$. Esto se relaciona con la prueba hecha con chocolate_sale_100_ptc.csv, donde se encontraron 49.241 pedidos para 70% Dark Bar con el filtro MENOR, y el tiempo de ejecución fue de 75.494 milisegundos. Al usar el archivo más grande, se puede ver cómo se comporta el requerimiento cuando tiene que revisar una cantidad alta de datos.

Requerimiento Ejemplo

Descripción

```
def get_data(data_structs, id):  
    """  
    Retorna un dato a partir de su ID  
    """  
    pos_data = lt.isPresent(data_structs["data"], id)  
    if pos_data > 0:  
        data = lt.getElement(data_structs["data"], pos_data)  
        return data  
    return None
```

Este requerimiento se encarga de retornar un dato de una lista dado su ID. Lo primero que hace es verificar si el elemento existe. Dado el caso que exista, retorna su posición, lo busca en la lista y lo retorna. De lo contrario, retorna None.

Entrada	Estructuras de datos del modelo, ID.
Salidas	El elemento con el ID dado, si no existe se retorna None
Implementado (Sí/No)	Si. Implementado por Juan Andrés Ariza

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Buscar si el elemento existe (isPresent)	$O(n)$
Obtener el elemento (getElement)	$O(1)$
TOTAL	$O(n)$

Análisis

A pesar de que obtener un elemento en un *ArrayList*, dada su posición, tiene complejidad constante, la implementación de este requerimiento tiene un orden lineal $O(n)$. Esto debido a que, lo primero que se hace es verificar si el elemento hace parte de la lista. Específicamente, a la hora de buscar un elemento en una lista, en el peor de los casos es necesario recorrer toda la lista, es decir, complejidad lineal.